

Part A Cheatsheet: Theory, Manual Calculation, Python Output

Focus: Week 8-13. Week 1-7 may appear, so keep the bottom safety net. Print two-sided, narrow margins, small font.

```
<style> body { font-size: 9px; line-height: 1.12; } table { font-size: 8px; } pre, code { font-size: 8px; } h1, h2, h3 { margin: 4px 0; } p, ul { margin: 3px 0; } </style>
```

Page 1: Week 8 to Week 13 Core

W8 Tree-Based Methods

Method	Theory	Command pattern	Interpret output
Classification tree	Splits predictor space; leaf predicts majority class.	<code>DecisionTreeClassifier(max_depth=d).fit(X_train,y_train)</code>	<code>.score</code> = accuracy. Deep tree = flexible, overfit risk.
Regression tree	Leaf predicts mean response.	<code>DecisionTreeRegressor(max_depth=d)</code>	<code>MSE</code> lower better; <code>RMSE=sqrt(MSE)</code> in response units.
Bagging	Bootstrap many trees, average/vote; reduces variance.	<code>RandomForestRegressor(max_features=p)</code>	If <code>max_features=all predictors</code> , it is bagging.
Random forest	Bagging + random predictors at each split.	<code>RandomForestRegressor(max_features=m)</code>	Often lower test error; <code>feature_importances_</code> ranks variables.
Boosting	Trees built sequentially from previous errors.	<code>GradientBoostingRegressor(n_estimators=500, learning_rate=.01)</code>	Accurate but can overfit if too many trees/high learning rate.

Commands:

```
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor,
GradientBoostingRegressor
tree = DecisionTreeClassifier(max_depth=6).fit(X_train,y_train)
pred = tree.predict(X_test); print(confusion_matrix(y_test,pred));
print(classification_report(y_test,pred))
rf = RandomForestRegressor(max_features=4, random_state=2).fit(X_train,y_train)
print(mean_squared_error(y_test, rf.predict(X_test))); print(rf.feature_importances_)
```

W9 Support Vector Machines

Item	Meaning	Exam interpretation
Hyperplane	Decision boundary.	Linear SVM uses straight boundary.
Margin	Distance from boundary to closest points.	Larger margin usually generalizes better.
Support vectors	Closest points determining boundary.	They control the classifier.
C	Cost of misclassification.	Small C: wider margin, more train errors, less flexible. Large C: fewer train errors, overfit risk.
gamma	RBF locality.	Large gamma: wiggly/local boundary, overfit risk.
Kernel	Shape of boundary.	linear, rbf, poly. Use RBF for nonlinear boundary.

Commands:

```
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV
svc = SVC(C=1, kernel="linear").fit(X_train,y_train)
print(confusion_matrix(y_test, svc.predict(X_test))); print(svc.score(X_test,y_test))
params = [{"C": [.01, .1, 1, 10, 100], "gamma": [.5, 1, 2, 3, 4]}]
clf = GridSearchCV(SVC(kernel="rbf"), params, cv=10, scoring="accuracy").fit(X_train,y_train)
print(clf.best_params_, clf.best_score_, clf.best_estimator_.score(X_test,y_test))
```

ROC/AUC: curve near top-left is good; AUC=1 perfect, 0.5 random. Train high + test low = overfit.

W10 PCA / Unsupervised Learning

Term	Meaning	Interpret output
Unsupervised	Only X, no response y.	Goal: structure, visualization, clusters.
PCA	New variables = linear combinations of original predictors.	PC1 explains most variance, PC2 next, PCs uncorrelated.
Scores	Observation coordinates on PCs.	Used to plot observations. Nearby = similar in PC space.
Loadings	Variable weights in each PC.	Large absolute loading = variable contributes strongly. Sign = direction.
PVE	Proportion variance explained.	0.62 means 62% of variance explained by that PC.
Scree plot	PVE vs PC number.	Look for elbow; cumulative PVE for total captured.

Commands:

```
from sklearn.preprocessing import scale
from sklearn.decomposition import PCA
X_scaled = pd.DataFrame(scale(df), index=df.index, columns=df.columns)
pca = PCA(); scores = pca.fit_transform(X_scaled)
loadings = pd.DataFrame(pca.components_.T, index=df.columns)
pve = pca.explained_variance_ratio_; cum_pve = np.cumsum(pve)
```

Always scale before PCA when variables have different units.

W11 Clustering: K-Means, K-Modes, K-Medoids, K-Prototypes

Method	Data	Center	Assignment	Output
K-means	Numerical	Mean centroid	Smallest distance	<code>.labels_</code> , <code>.cluster_centers_</code> , <code>.inertia_</code>
K-modes	Categorical	Mode	Lowest mismatch count	<code>.labels_</code> , <code>.cluster_centroids_</code> , <code>.cost_</code>
K-medoids	Numerical/dissimilarity	Actual observation	Nearest medoid	Robust to outliers
K-prototypes	Mixed	Mean + mode	Combined distance	Use categorical column index

Commands:

```
from sklearn.cluster import KMeans
from kmodes.kmodes import KModes
from kmodes.kprototypes import KPrototypes
kmeans = KMeans(n_clusters=3, n_init=20, random_state=123).fit(X)
print(kmeans.labels_, kmeans.cluster_centers_, kmeans.inertia_) # lower inertia better
km = KModes(n_clusters=4, init="Huang", n_init=5).fit(data)
print(km.labels_, km.cluster_centroids_, km.cost_) # lower cost better
kproto = KPrototypes(n_clusters=3, init="Cao")
clusters = kproto.fit_predict(customers_norm, categorical=[0,1])
```

Manual calculation:

Euclidean = $\sqrt{\text{sum squared differences}}$; Manhattan = sum absolute differences.
 K-means centroid = mean of each variable in cluster.
 K-modes dissimilarity: same=0, different=1, sum scores.
 K-modes center = most frequent category per variable.
 Assign to smallest distance/dissimilarity. Recalculate center. Stop when labels do not change.

K-means template:

$CA_{x1} = (\text{x1 values in A})/n$, $CA_{x2} = (\text{x2 values in A})/n$, ...
 $d(O_i, CA) = \sqrt{(x_{1i} - CA_1)^2 + (x_{2i} - CA_2)^2 + \dots}$
 nearest centroid wins.

K-modes template:

$O = (A, L, M)$, $C = (A, L, C)$: $A=A \rightarrow 0$, $L=L \rightarrow 0$, $M=C \rightarrow 1$, total=1.
 lowest total wins. If tie, state tie rule.

W12 Hierarchical Clustering and EM

Topic	Meaning	Interpret
Hierarchical	Nested clusters, dendrogram.	No need to choose K first.
Agglomerative	Bottom-up merging.	Start each point as own cluster.
Dendrogram height	Dissimilarity at merge.	Lower merge = more similar; vertical axis matters, not horizontal order.

Topic	Meaning	Interpret
Complete linkage	Farthest pair between clusters.	Compact clusters.
Single linkage	Closest pair.	Can create chaining/trailing clusters.
Average linkage	Average pairwise distance.	Middle behavior.
EM / GMM	Soft clustering.	E-step membership probability, M-step update parameters.

Commands:

```
from scipy.cluster.hierarchy import linkage, dendrogram, cut_tree
hc = linkage(X, method="complete", metric="euclidean")
dendrogram(hc); labels = cut_tree(hc, n_clusters=4).reshape(-1)
from sklearn.mixture import GaussianMixture
gmm = GaussianMixture(n_components=2).fit(X)
print(gmm.predict(X)); print(gmm.predict_proba(X))
```

W13 Neural Networks

Term	Meaning	Interpret
Perceptron	One neuron.	Only simple/linear boundaries.
MLP	Multiple hidden layers.	Can model nonlinear data.
Feed-forward	Inputs -> weights + bias -> activation -> output.	Prediction.
Backpropagation	Updates weights from error.	Training.
Activation	ReLU/tanh/sigmoid nonlinear function.	Gives flexibility.
Scaling	Standardize features.	Very important for NN training.
<code>hidden_layer_sizes</code>	Architecture.	<code>(10,10,10)</code> = 3 hidden layers, 10 nodes each.

Commands:

```
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.neural_network import MLPClassifier, MLPRegressor
scaler = StandardScaler(); X_train=scaler.fit_transform(X_train); X_test=scaler.transform(X_test)
mlp = MLPClassifier(hidden_layer_sizes=(10,10,10),
max_iter=1000).fit(X_train,y_train.values.ravel())
pred = mlp.predict(X_test); print(confusion_matrix(y_test,pred));
print(classification_report(y_test,pred))
reg = MLPRegressor(activation="relu", hidden_layer_sizes=(16,),
random_state=0).fit(X_train,y_train)
print(mean_squared_error(y_test,reg.predict(X_test)), r2_score(y_test,reg.predict(X_test)))
```

Page 2: Output Interpretation, Formulas, Week 1-7 Safety Net

Python Output Interpretation

Output	Meaning	What to say
<code>.score()</code> classifier	Accuracy.	Higher test accuracy better.
<code>.score()</code> regressor	Usually R^2 .	1 perfect, 0 like mean, negative poor.
Confusion matrix	Correct/wrong counts.	Diagonal correct; off-diagonal errors.
Precision	$TP/(TP+FP)$	Low precision = many false positives.
Recall / sensitivity	$TP/(TP+FN)$	Low recall = many false negatives.
F1	Balance of precision and recall.	Useful for imbalanced classes.
Support	Actual count per class.	Check class imbalance.
MSE	Average squared error.	Lower better.
RMSE	$\sqrt{MSE}$	Error in original response units.
<code>coef_</code> / coefficient	Effect of predictor.	Sign gives direction; size depends on scale.
<code>p></code>	t	
R-squared	Variance explained.	Higher fit, but can rise with more predictors.
Adjusted R2	R2 penalized for predictors.	Better for model comparison.
AIC/BIC	Selection criteria.	Lower better; BIC penalizes complexity more.
CV score/error	Estimated test performance.	Use to choose model.

Confusion matrix:

```

          Pred 0   Pred 1
Actual 0      TN      FP
Actual 1      FN      TP
Accuracy=(TP+TN)/total; Precision=TP/(TP+FP); Recall=TP/(TP+FN)

```

Core Formulas

```

Linear regression:  $y = b_0 + b_1x_1 + \dots + b_px_p + e$ 
Residual = actual - predicted;  $RSS = \sum(\text{residual}^2)$ ;  $MSE = \text{mean}(\text{residual}^2)$ ;  $RMSE = \sqrt{MSE}$ 
Logistic:  $p = \exp(b_0 + b_1x) / (1 + \exp(b_0 + b_1x))$ ;  $\text{odds} = p / (1 - p)$ ;  $\log\text{-odds} = b_0 + b_1x$ 
Euclidean =  $\sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2 + \dots}$ 
Manhattan =  $|a_1 - b_1| + |a_2 - b_2| + \dots$ 
PVE = variance explained by PC / total variance
K-means inertia = sum squared distance to assigned centroid

```

Week 1-7 Safety Net

Week	Key theory	Command/output
W1 Intro	Regression = quantitative y ; classification = categorical y ; clustering/PCA = no y . Train data fits model, test data estimates future performance.	<code>train_test_split</code> , <code>LinearRegression</code> , <code>KMeans</code> , <code>PCA</code>

Week	Key theory	Command/output
W2 Linear regression	Coefficients: sign/direction. p-value tests relationship. Check residuals/nonlinearity/outliers/leverage. Polynomial regression is still linear in coefficients.	<code>smf.ols("y ~ x1+x2",data=df).fit().summary()</code>
W3 Classification	Logistic predicts probability; threshold gives class. LDA assumes Gaussian classes/common covariance.	<code>LogisticRegression(), LinearDiscriminantAnalysis(), confusion_matrix()</code>
W4 Resampling	Validation set, LOOCV, k-fold CV estimate test error. Bootstrap samples with replacement to estimate uncertainty.	<code>cross_val_score, LeaveOneOut, resample</code>
W5 Selection/regularization	Best subset checks all models; forward/backward search. Ridge shrinks coefficients; lasso can set coefficients to zero.	<code>RidgeCV, LassoCV</code> ; lower CV error/AIC/BIC better
W6 PCR/PLS	PCR = PCA then regression; choose number of PCs by CV. PLS is supervised and uses <code>y</code> .	<code>PCA, LinearRegression</code> , CV MSE
W7 Nonlinear	Polynomial, step functions, splines, local regression, GAMs increase flexibility. Choose complexity by p-values/CV.	<code>PolynomialFeatures, pd.cut, dmatrix("bs(...)", anova_lm</code>

Common Commands

```
# split + metrics
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.metrics import confusion_matrix, classification_report, mean_squared_error, r2_score
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=.2,random_state=0)

# statsmodels regression
import statsmodels.formula.api as smf
res = smf.ols("medv ~ lstat + rm", data=df).fit(); print(res.summary())

# sklearn regression/classification
from sklearn.linear_model import LinearRegression, LogisticRegression, RidgeCV, LassoCV
lr = LinearRegression().fit(X_train,y_train); print(lr.coef_, lr.intercept_)
logreg = LogisticRegression(C=100000, tol=1e-7).fit(X_train,y_train)

# CV
scores = cross_val_score(model, X, y, cv=10, scoring="accuracy")
errors = -cross_val_score(model, X, y, cv=10, scoring="neg_mean_squared_error")

# regularization
alphas = np.logspace(-3,3,100)
ridge = RidgeCV(alphas=alphas).fit(X_train,y_train)
lasso = LassoCV(alphas=alphas, max_iter=100000).fit(X_train,y_train)
print(ridge.alpha_, lasso.alpha_, lasso.coef_) # lasso zeros = removed variables

# nonlinear
from sklearn.preprocessing import PolynomialFeatures
X4 = PolynomialFeatures(degree=4).fit_transform(X)
from patsy import dmatrix
spl = dmatrix("bs(age, df=6, include_intercept=False)", {"age":df.age}, return_type="dataframe")
```

Fast Comparisons

Compare	Short answer
Training vs test error	Training is optimistic; test estimates unseen performance.
Bias vs variance	Simple: high bias/low variance. Flexible: low bias/high variance.
Bagging vs RF	Bagging uses all predictors per split; RF uses random subset.
RF vs boosting	RF independent trees; boosting sequential trees.
PCA vs clustering	PCA reduces/visualizes; clustering groups observations.
K-means vs hierarchical	K-means needs K ; hierarchical uses dendrogram then cut.
K-means vs K-modes	Numerical mean vs categorical mode.
Hard vs soft clustering	One cluster label vs membership probabilities.
Ridge vs lasso	Ridge shrinks; lasso shrinks and selects variables.
PCR vs PLS	PCR ignores y when creating components; PLS uses y .

Answer Templates

Output interpretation: Test ____ is _____. This means _____. Compared with _____, it is better/worse because _____. If train much better than test, possible overfitting.

Model selection: choose lower test/CV MSE or higher test/CV accuracy/F1; for AIC/BIC choose lower.

K-means: compute centroids -> distances -> nearest cluster -> update centroids -> stop if unchanged.

K-modes: same=0/different=1 -> sum -> lowest cluster -> update mode -> mention ties.